

Implementation Phases Overview

Christmas Tracker iOS Application

Version: 1.0
Date: February 4, 2026
Total Phases: 13
Estimated Total Effort: 100-128k tokens

Executive Summary

This document outlines the complete implementation roadmap for the Christmas Tracker iOS application. The project is structured into 13 phases, progressing from foundation through deployment. Each phase builds upon previous phases, with clear dependencies and deliverables.

MVP Cutline: End of Phase 8 (Profile & Settings)
v1.1 Features: Phases 9-10 (Share Extension, Version Management, Push Notifications)
Polish & Launch: Phases 11-13 (Polish, Testing, CI/CD)

Phase Overview

Phase	Name	Effort	Status	Dependencies
1	Foundation & Architecture	8-10k	✔ Complete	None
2	Network & Security Infrastructure	10-12k	Planned	Phase 1
3	Authentication Flows	8-10k	Planned	Phase 2
4	Navigation & Dashboard Shell	6-8k	Planned	Phase 3
5	List Management	10-12k	Planned	Phase 4
6	Item Management	12-14k	Planned	Phase 5
7	Statistics & Budget	6-8k	Planned	Phase 6
8	Profile & Settings	6-8k	Planned	Phase 3
MVP CUTLINE				
9	Share Extension	10-12k	Planned	Phase 6
9.5	Version Management	8-10k	Planned	Phase 3
10	Push Notifications	8-10k	Planned	Phase 8
11	Polish & Optimization	6-8k	Planned	All
12	Testing & CI/CD	6-8k	Planned	All

Total Estimated Effort: 104-130k tokens

Phase 1: Foundation & Architecture

Effort: 8-10 hours
Status: ✔ Complete

Objectives

Set up the complete project foundation with iOS 26, Swift 6, and all architectural patterns in place.

Deliverables

-  Xcode project configured (iOS 26, Swift 6, strict concurrency)
-  xcconfig files (Dev, QA, Production) with API base URLs
-  Complete folder structure (Features, Core, DesignSystem, etc.)
-  Design System in Asset Catalog (colors, typography, spacing)
-  AppConfig reading from xcconfig
-  Core protocol definitions (APIClient, Repository, LogDestination)
-  Mock data factory for previews and tests
-  First SwiftUI preview (DashboardView shell)
-  First unit tests passing (AppConfiguration)
-  Git repository initialized

Key Decisions

- Service-Oriented Architecture with Observation framework
- Domain-partitioned DataStores (no god objects)
- Repository pattern with protocol-based design
- Fire-and-forget logging (non-blocking)
- 3 environments (Dev, QA, Production)

Phase 2: Network & Security Infrastructure

Effort: 10-12 hours

Dependencies: Phase 1

Objectives

Implement core networking, caching, security, and logging infrastructure.

Deliverables

- Actor-based APIClient with automatic retry and error handling
- NetworkCache with session-scoped, selective eviction (`.none`, `.lists`, `.items`, `.all`)
- KeychainService with biometric protection and Secure Enclave
- DeviceFingerprintService (UUID + model + version)
- Logger actor with multiple destinations:
 - New Relic SDK (if Swift 6 compatible)
 - Backend destination (fallback via `/logs/batch`)
 - OSLog (iOS native, Instruments integration)
 - ConsoleDestination (debug builds only)
- PIIFilter for automatic redaction (emails, tokens, SSN, etc.)
- ErrorInterceptor for global 401/426 handling
- Signpost support for performance tracking
- Repository implementations (Network + Mock for each domain)

Key Files

- `Core/Networking/APIClient.swift`
- `Core/Networking/NetworkCache.swift`
- `Core/Security/KeychainService.swift`
- `Core/Security/DeviceFingerprintService.swift`
- `Core/Logging/Logger.swift`
- `Core/Logging/PIIFilter.swift`

Testing

- Unit tests for cache eviction logic
 - Unit tests for keychain operations
 - Unit tests for PII filtering
 - Integration tests for API client
-

Phase 3: Authentication Flows

Effort: 8-10 hours

Dependencies: Phase 2

Objectives

Implement complete authentication system with password, biometric, and keep-alive session management.

Deliverables

- AuthenticationService with login, register, biometric, refresh, logout
- SessionManager for keep-alive (30min base, 5min ping, 24h max)
- Login screen (email/password, "Remember me")
- Register screen (email, password, name)
- Biometric setup prompt (after first login)
- Token refresh logic with rotation
- Session activity tracking (view modifier)
- "Forgot Password" flow
- Backend integration:
 - `POST /auth/login` (with version headers)
 - `POST /auth/register`
 - `POST /auth/biometric` (iOS-specific)
 - `POST /auth/refresh` (token rotation)
 - `POST /auth/ping` (session extension)
 - `POST /auth/logout`

Key Files

- `Features/Authentication/Services/AuthenticationService.swift`
- `Features/Authentication/Services/SessionManager.swift`
- `Features/Authentication/Views/LoginView.swift`
- `Features/Authentication/Views/RegisterView.swift`
- `Features/Authentication/Views/BiometricSetupView.swift`

Testing

- Unit tests for authentication flows
 - Unit tests for session management
 - UI tests for login/register flows
 - Integration tests for biometric authentication
-

Phase 4: Navigation & Dashboard Shell

Effort: 6-8 hours

Dependencies: Phase 3

Objectives

Implement tab-based navigation and dashboard with empty states.

Deliverables

- Tab bar (Dashboard, Stats, Profile)
- Dashboard screen with pull-to-refresh
- "Last updated" indicator for stale data
- Empty states for all tabs
- Navigation coordinator for deep linking
- Loading states + skeleton views
- Background refresh logic (>15 min threshold)

Key Files

- `Navigation/TabBarView.swift`
- `Navigation/NavigationCoordinator.swift`
- `Features/Dashboard/Views/DashboardView.swift`
- `DesignSystem/Components/Loading/SkeletonView.swift`
- `DesignSystem/Components/EmptyStates/EmptyStateView.swift`

Testing

- UI tests for tab navigation
- UI tests for pull-to-refresh
- Unit tests for background refresh logic

Phase 5: List Management

Effort: 10-12 hours

Dependencies: Phase 4

Objectives

Implement complete list CRUD operations with permissions and invitations.

Deliverables

- ListService + ListRepository + ListDataStore integration
- List CRUD screens (create, edit, delete)
- Swipe-to-delete with confirmation
- List detail view (members, items preview, stats)
- List permissions UI (owner/co-owner/member/viewer badges)
- Invitation flow:
 - Send invitation (email + role selection)
 - Accept invitation
 - Decline invitation
 - Pending invitations list
- List filtering/sorting (owned, shared, all)
- List search
- Backend integration:
 - `GET /lists` (with role filtering)
 - `POST /lists`
 - `PUT /lists/:id`
 - `DELETE /lists/:id`
 - `POST /lists/:id/invitations`
 - `POST /invitations/:id/accept`

- `POST /invitations/:id/decline`

Key Files

- `Features/Lists/Services/ListService.swift`
- `Features/Lists/DataStores/ListDataStore.swift`
- `Features/Lists/Views/ListDetailView.swift`
- `Features/Lists/Views/CreateListView.swift`
- `Features/Lists/Views/InvitationView.swift`

Testing

- Unit tests for list service
- Unit tests for invitation logic
- UI tests for list CRUD flows
- UI tests for invitation flows

Phase 6: Item Management

Effort: 12-14 hours

Dependencies: Phase 5

Objectives

Implement complete item CRUD operations with purchase tracking and off-list items.

Deliverables

- `ItemService` + `ItemRepository` + `ItemDataStore` integration
- Item CRUD screens (add, edit, delete)
- Item detail view (name, description, URL, image, price, quantity)
- Purchase flow:
 - Mark as purchased (optimistic concurrency)
 - Undo purchase
 - Purchase confirmation modal
 - Updates list stats automatically
- Off-list "surprise gift" functionality:
 - Hidden from list owner
 - Only visible to purchaser
 - Special badge/indicator
- Item quantity management (stepper)
- Price input with validation (decimal, currency formatting)
- URL preview (fetch metadata)
- Image handling (`AsyncImage` with cache)
- Backend integration:
 - `GET /lists/:id/items`
 - `POST /lists/:id/items`
 - `PUT /items/:id`
 - `DELETE /items/:id`
 - `POST /items/:id/purchase`
 - `DELETE /items/:id/purchase` (undo)

Key Files

- `Features/Items/Services/ItemService.swift`
- `Features/Items/DataStores/ItemDataStore.swift`

- `Features/Items/Views/ItemDetailView.swift`
- `Features/Items/Views/AddItemView.swift`
- `Features/Items/Views/PurchaseItemView.swift`

Testing

- Unit tests for item service
- Unit tests for purchase logic
- Unit tests for optimistic concurrency
- UI tests for item CRUD flows
- UI tests for purchase flows

Phase 7: Statistics & Budget

Effort: 6-8 hours

Dependencies: Phase 6

Objectives

Implement budget tracking and visualization with Swift Charts.

Deliverables

- `StatisticsService` + `StatisticsRepository`
- Budget tracking charts:
 - Per-list spending (bar chart)
 - Per-person spending (pie chart)
 - Category breakdown (if implemented)
 - Timeline view (spending over time)
- Budget limit warnings:
 - Soft limit (yellow warning at 80%)
 - Confirmation dialog if exceeding budget
 - Override option
- Spending breakdown (purchased vs remaining)
- Filters:
 - Date range (this year, last year, all time)
 - List selection
 - Person selection
- Export data:
 - CSV format
 - Share sheet integration
- Backend integration:
 - `GET /statistics/lists/:id`
 - `GET /statistics/users/:id`
 - `GET /statistics/summary`

Key Files

- `Features/Statistics/Services/StatisticsService.swift`
- `Features/Statistics/Views/StatisticsView.swift`
- `Features/Statistics/Views/BudgetChartView.swift`
- `Features/Statistics/Views/SpendingChartView.swift`

Testing

- Unit tests for statistics calculations

- Unit tests for budget limit logic
 - UI tests for chart interactions
 - UI tests for export functionality
-

Phase 8: Profile & Settings

Effort: 6-8 hours

Dependencies: Phase 3

Objectives

Implement user profile management, settings, and appearance controls.

Deliverables

- ProfileService + ProfileRepository
- User profile editing:
 - Name
 - Email (with verification)
 - Avatar (photo picker + upload)
- Account management:
 - Change password
 - Delete account (with confirmation)
- Notification preferences UI:
 - Per-device toggles
 - Sync with backend
- Biometric settings:
 - Enable/disable Face ID/Touch ID
 - Re-authentication prompt
- Appearance settings:
 - AppearanceManager (@Observable)
 - Light/Dark/System mode picker
 - Persisted to UserDefaults
- About/Help screens:
 - App version display
 - Privacy policy link
 - Terms of service link
 - Contact support
- Backend integration:
 - GET /profile
 - PUT /profile
 - POST /profile/avatar
 - POST /auth/change-password
 - DELETE /account

Key Files

- Features/Profile/Services/ProfileService.swift
- Features/Profile/Views/ProfileView.swift
- Features/Profile/Views/EditProfileView.swift
- Features/Profile/Views/AppearanceSettingsView.swift
- Core/Configuration/AppearanceManager.swift

Testing

- Unit tests for profile service

- UI tests for profile editing
 - UI tests for appearance toggle
-

MVP CUTLINE

Phases 1-8 constitute the Minimum Viable Product (MVP).

At this point, the app is feature-complete for core functionality:

-  User authentication (password + biometric)
-  List management with permissions
-  Item management with purchase tracking
-  Budget tracking and visualization
-  User profile and settings
-  Dark mode support

Decision Point: Evaluate whether to proceed with v1.1 features or release MVP first.

Phase 9: Share Extension

Effort: 10-12 hours

Dependencies: Phase 6

Objectives

Enable users to add gift ideas directly from Safari or other apps via iOS Share Extension.

Deliverables

- Share extension target setup
- Shared keychain group configuration (auth token access)
- Two-step list selection UI:
 - Step 1: Mode picker ("My Lists" vs "Someone's List")
 - Step 2: List picker (filtered by mode)
- URL metadata extraction:
 - iOS attempt (LPMetadataProvider, 5 sec timeout)
 - Fallback to backend if timeout
 - Extract: title, description, imageUrl, price
- Pending items flow:
 - "Save Now" (complete in extension)
 - "Save as Pending" (complete in main app)
 - Pending items hidden from other list members
- Extension → app handoff (deep link to pending items)
- Backend integration:
 - `POST /share/item` (with isPending flag)
 - `GET /share/lists?type=owned|member` (lightweight)

Key Files

- `ShareExtension/ShareViewController.swift`
- `ShareExtension/Views/ShareFormView.swift`
- `ShareExtension/Services/ShareService.swift`
- `ShareExtension/Utilities/URLMetadataExtractor.swift`

Backend Changes Required

- Add `isPending` boolean to Item model
- Filter pending items from list queries
- Add share endpoints (lightweight, no heavy business logic)

Testing

- UI tests for share extension flows
- Integration tests for metadata extraction
- UI tests for pending items in main app

Phase 9.5: Version Management

Effort: 8-10 hours

Dependencies: Phase 3

Objectives

Ensure users stay up-to-date with version checking and enforcement.

Deliverables

Backend:

- Database schema: `AppVersionPolicy` table
- User model updates: `lastAppVersion`, `lastAppPlatform`
- New endpoint: `GET /app/version-check`
- Modified endpoints: `POST /auth/login` and `POST /auth/refresh` return 426 if outdated
- All API requests validate `X-App-Version` and `X-Platform` headers
- Logging enhancement: Include app version in all logs

iOS:

- VersionManager actor:
 - Launch check (always on cold start)
 - Resume check (if > 15 min background)
 - Periodic check (every 6 hours in-session)
 - Handles: ok, update_recommended, force_update
- APIClient enhancement:
 - Add version/platform headers to all requests
 - Handle 426 globally
- Update prompt UI:
 - Gentle nudge (dismissible)
 - Force update (blocking with "Update Now" or "Logout")
- App lifecycle integration
- Logging enhancement: Include version in client logs

Key Files

- `Core/Versioning/VersionManager.swift`
- `Features/Versioning/UpdatePromptView.swift`
- Backend: `app_version_policies` table
- Backend: `GET /app/version-check`, modified auth endpoints

Backend Changes Required

- Version policy database table
- Version check endpoint

- 426 response on outdated versions
- Version tracking in User model

Testing

- Unit tests for version comparison logic
- UI tests for update prompts
- Integration tests for 426 handling
- Backend tests for version enforcement

Phase 10: Push Notifications

Effort: 8-10 hours

Dependencies: Phase 8

Objectives

Implement push notifications for key events with per-device preferences.

Deliverables

- APNs setup (certificates, entitlements, capabilities)
- NotificationService:
 - Register device token
 - Handle incoming notifications
 - Deep link routing
- Device notification preferences:
 - Item added to co-owned list
 - Watched item purchased
 - Budget limit approaching
 - Time-based reminders
 - New list created
 - Item added to member list
- Rich notifications (images, actions)
- Notification settings UI in Profile
- Deep linking:
 - Universal Links setup (if deployment supports)
 - Fallback to custom URL scheme (`christmastracker://`)
- Backend integration:
 - `POST /notifications/register`
 - `PUT /notifications/preferences`
 - `DELETE /notifications/unregister`

Key Files

- `Features/Notifications/Services/NotificationService.swift`
- `Features/Notifications/Services/DeepLinkHandler.swift`
- `Features/Profile/Views/NotificationSettingsView.swift`

Backend Changes Required

- DeviceNotificationSettings table
- APNs integration
- Notification scheduling logic
- Deep link URL generation

Testing

- Unit tests for notification handling
 - Unit tests for deep link parsing
 - Integration tests for APNs registration
 - UI tests for notification settings
-

Phase 11: Polish & Optimization

Effort: 6-8 hours

Dependencies: All previous phases

Objectives

Add polish, optimize performance, and enhance user experience.

Deliverables

- Haptic feedback:
 - Success (purchase, create)
 - Error (failed action)
 - Selection (picker, toggle)
- Loading states refinement:
 - Skeleton views for all lists
 - Progressive loading
 - Optimistic UI updates
- Error illustrations:
 - Custom illustrations for error states
 - Friendly error messages
- Empty state refinement:
 - Contextual messages
 - Actionable CTAs
 - Custom illustrations
- Accessibility:
 - VoiceOver labels on all interactive elements
 - Dynamic Type support (test at largest size)
 - Contrast validation (WCAG AA minimum)
 - Touch targets (44x44pt minimum)
- Performance optimization:
 - List virtualization (LazyVStack/LazyVGrid)
 - Image caching strategy
 - Lazy loading for large lists
 - Memory profiling with Instruments
- Animations:
 - Page transitions
 - Button press feedback
 - List item animations
 - Modal presentations
- App icon (1024x1024, all required sizes)
- Launch screen design

Key Files

- All views updated with haptics
- All views updated with accessibility
- `DesignSystem/Components/Loading/` refinements
- `DesignSystem/Components/EmptyStates/` refinements

Testing

- Accessibility audit (VoiceOver, Dynamic Type)
 - Performance testing (Instruments)
 - Memory leak detection
 - Battery usage profiling
-

Phase 12: Testing & CI/CD

Effort: 6-8 hours

Dependencies: All previous phases

Objectives

Achieve 80% test coverage and set up continuous integration.

Deliverables

- Unit test suite:
 - Services (auth, list, item, statistics, profile)
 - Repositories (network and cache logic)
 - DataStores (state management)
 - ViewModels (if any)
 - Target: 80% coverage for business logic
- XCUITest suite:
 - Critical flows:
 - Login flow (password + biometric)
 - Create list flow
 - Add item flow
 - Purchase item flow
 - Smoke tests for all major screens
- Test data seeding:
 - Scripts to populate test database
 - Mock data factories
- GitHub Actions workflow:
 - Run tests on PR to main/release branches
 - Fail build on test failure
 - Generate code coverage report
 - Upload coverage to Codecov
- Sentry crash reporting:
 - SDK integration
 - User context attachment
 - Release health tracking
- CI/CD pipeline validation:
 - Build succeeds on all configurations
 - Tests pass on simulator
 - Linting passes (SwiftLint)

Key Files

- `.github/workflows/ios.yml`
- `ChristmasTrackerTests/` (all test files)
- `ChristmasTrackerUITests/` (all UI test files)
- `Scripts/seed-test-data.sh`

Coverage Goals

- **Business logic:** 80%+ (services, repositories, datastores)
- **UI code:** Not unit tested (covered by UI tests)
- **Critical flows:** 100% UI test coverage

Testing

- All tests must pass before merge
- Coverage report generated on each PR
- Failed tests block deployment

 v1.0 Release Candidate

After Phase 12, the app is ready for TestFlight beta testing.

Key Milestones

Milestone	Phases	Description	Target Date
Foundation Complete	1-2	Core architecture in place	Week 2
Authentication Working	3	Users can login/register	Week 3
Core Features Complete	4-6	Lists and items functional	Week 6
MVP Complete	7-8	All core features + settings	Week 8
v1.1 Features	9-10	Share extension + notifications	Week 11
Polish & Testing	11-12	Production-ready	Week 13
Beta Release	-	TestFlight distribution	Week 14
App Store Launch	-	Public availability	Week 16

Dependencies & Blockers

External Dependencies

- **Backend API:** All endpoints must be available before phase implementation
- **App Store Listing:** Required for Phase 9.5 (update URL)
- **APNs Certificate:** Required for Phase 10 (push notifications)
- **New Relic Account:** Required for Phase 2 (logging)

Internal Dependencies

- **Design Assets:** App icon, launch screen (Phase 11)
- **Legal:** Privacy policy, terms of service (Phase 8)
- **Content:** Help documentation (Phase 8)

Known Risks

1. **New Relic Swift 6 compatibility** - Phase 2
 - Mitigation: Backend fallback already designed
 2. **Apple review rejection** - Version management too aggressive
 - Mitigation: Use force update only for critical security issues
 3. **Universal Links setup** - Deployment infrastructure required
 - Mitigation: Custom URL scheme fallback
-

Backend API Requirements Summary

Phase 2: Network Infrastructure

- No new endpoints (infrastructure only)

Phase 3: Authentication

-  **POST** /auth/login (add deviceFingerprint, appVersion, platform)
-  **POST** /auth/register
-  **POST** /auth/biometric (new, iOS-specific)
-  **POST** /auth/refresh (add token rotation)
-  **POST** /auth/ping (new, session keep-alive)
-  **POST** /auth/logout

Phase 5: Lists

-  **GET** /lists
-  **POST** /lists
-  **GET** /lists/:id
-  **PUT** /lists/:id
-  **DELETE** /lists/:id
-  **POST** /lists/:id/invitations
-  **POST** /invitations/:id/accept
-  **POST** /invitations/:id/decline

Phase 6: Items

-  **GET** /lists/:id/items
-  **POST** /lists/:id/items
-  **GET** /items/:id
-  **PUT** /items/:id
-  **DELETE** /items/:id
-  **POST** /items/:id/purchase
-  **DELETE** /items/:id/purchase

Phase 7: Statistics

-  **GET** /statistics/lists/:id
-  **GET** /statistics/users/:id
-  **GET** /statistics/summary

Phase 8: Profile

-  **GET** /profile
-  **PUT** /profile
-  **POST** /profile/avatar
-  **POST** /auth/change-password
-  **DELETE** /account

Phase 9: Share Extension

-  **POST** /share/item (with isPending flag)
-  **GET** /share/lists?type=owned|member
-  Add **isPending** field to Item model

Phase 9.5: Version Management

-  GET /app/version-check
-  Modify POST /auth/login to return 426 if outdated
-  Modify POST /auth/refresh to return 426 if outdated
-  Add app_version_policies table
-  Add lastAppVersion, lastAppPlatform to User model

Phase 10: Push Notifications

-  POST /notifications/register
-  PUT /notifications/preferences
-  DELETE /notifications/unregister
-  Add DeviceNotificationSettings table
-  APNs integration

Phase 12: Testing

-  POST /logs/batch (only if New Relic incompatible)

Token Budget Tracking

Phase	Estimated	Actual	Notes
Phase 1	8-10k	10.2k	Complete
Phase 2	10-12k	-	Pending
Phase 3	8-10k	-	Pending
Phase 4	6-8k	-	Pending
Phase 5	10-12k	-	Pending
Phase 6	12-14k	-	Pending
Phase 7	6-8k	-	Pending
Phase 8	6-8k	-	Pending
Phase 9	10-12k	-	Pending
Phase 9.5	8-10k	-	Pending
Phase 10	8-10k	-	Pending
Phase 11	6-8k	-	Pending
Phase 12	6-8k	-	Pending
Total	104-130k	10.2k	7.8% Complete

Remaining Budget: ~73k tokens (sufficient for Phases 2-12)

Success Criteria

Phase Completion Criteria

Each phase is complete when:

-  All deliverables implemented
-  Unit tests passing (80% coverage for business logic)
-  UI tests passing for critical flows
-  Code review completed

-  Documentation updated
-  Committed to main branch

MVP Success Criteria (End of Phase 8)

-  All core features functional
-  80% test coverage achieved
-  No critical bugs
-  Performance targets met (app launch <2s, screen transitions <300ms)
-  Accessibility requirements met (VoiceOver, Dynamic Type, contrast)
-  Backend API stable and documented

v1.0 Launch Criteria (End of Phase 12)

-  All phases complete
-  Beta testing complete (50+ testers, 2+ weeks)
-  No critical or high-priority bugs
-  App Store assets ready (screenshots, descriptions, preview video)
-  Privacy policy and terms published
-  Support infrastructure ready (email, help docs)
-  Analytics and crash reporting configured

Next Steps

1. **Review this document** with team/stakeholders
2. **Prioritize any changes** to phase ordering
3. **Confirm backend API availability** for each phase
4. **Begin Phase 2** implementation (Network & Security Infrastructure)
5. **Generate detailed Phase 2 guide** when ready to start

Document History

Version	Date	Author	Changes
1.0	2026-02-04	Development Team	Initial phases overview created with Phase 9.5

End of Implementation Phases Overview